

Lecture 9: Maximum Independent Set of Rectangles - A Divide-and-Conquer Algorithm

Date: August 24, 2026

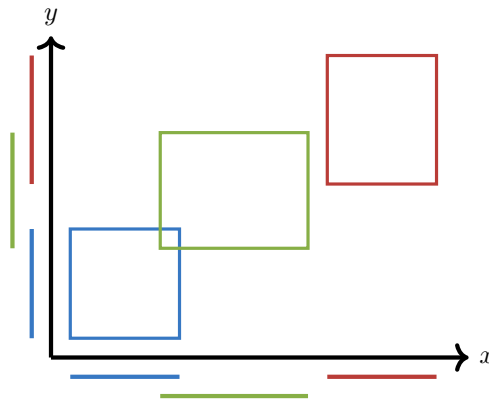
Instructor: Prof. Aniket Basu Roy

Scribe: Gandharva Dattaprasad Thite and Niranjan Mane

Overview. Every problem we study this semester will be treated in the same four steps: *formalize* the problem precisely, *design* an algorithm, *prove* the algorithm correct, and *analyze* its efficiency. Today we carry out all two steps in full on our problem, the MAXIMUM INDEPENDENT SET OF RECTANGLES PROBLEM.

1 Maximum Independent Set of Rectangles - A Divide-and-Conquer Algorithm

Motivation. Consider an interval scheduling problem extended to two dimensions: given a collection of axis-parallel rectangles $[x_1, x_2] \times [y_1, y_2]$, we wish to select the maximum number of non-overlapping rectangles.



Definition 1 (Instance). An instance consists of a family of axis-parallel rectangles $R = \{R_1, R_2, \dots, R_n\}$, where each rectangle is defined as $[x_{i,1}, x_{i,2}] \times [y_{i,1}, y_{i,2}]$.

Definition 2 (Independent Set). A subset $R' \subseteq R$ is an *independent set* if for all distinct pairs $R_i, R_j \in R'$, we have $R_i \cap R_j = \emptyset$.

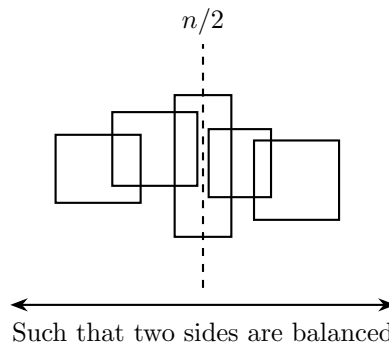
Objective: The *Maximum Independent Set of Rectangles* problem tries to find an independent set $R' \subseteq R$ that maximizes the cardinality $|R'|$.

Fact: This is an NP-Hard problem.

So if it's NP-Hard, maximizing $|R'|$ is of no use, we need to get at least a good solution so to obtain one, we can think of a Divide & Conquer solution.

Assumption: All the x_i, x_j are distinct & y_i, y_j are distinct.

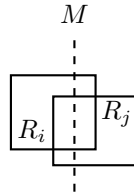
Maximum Independent Set of Rectangles - A Divide-and-Conquer Algorithm



$R_M = \{R \in R \mid R \cap M \neq \emptyset\}$ considering only the rectangles intersecting the median $(n/2)$ line.

Solving on R_M is equivalent to the 1-D Interval Scheduling problem.

For $R_i, R_j \in R_M$, if $R_i \cap R_j \neq \emptyset$:



$$\implies R_i^y \cap R_j^y \neq \emptyset$$

If two or more rectangles are intersecting on a median line, there “ y ” are also intersecting.

$$R = R_{\text{Left}} \cup R_{\text{Median}} \cup R_{\text{Right}}$$

Now recursive call will be on R_{Left} & R_{Right} .

$$R \longleftrightarrow V$$

$$R_i \cap R_j \neq \emptyset \longleftrightarrow (v_i, v_j) \in E$$

$$G = (V, E) \longleftrightarrow \text{Independent Set}$$

2 Implementation of Algorithm

Algorithm 1: Maximum Independent Set of Rectangles

Input: A family of rectangles $R = \{R_1, \dots, R_n\}$

Output: $R' \subseteq R$ such that $\forall R_i, R_j \in R'; R_i \cap R_j = \emptyset$

- 1 Independent Set (abbreviated to IS)(R):
 - 2 Find a median M , partition $R = R_{\text{Left}} \cup R_{\text{Median}} \cup R_{\text{Right}}$
 - 3 Solve $IS(R_{\text{Median}})$ as an interval scheduling problem on y -axis
 - 4 Solve $IS(R_{\text{Left}}) \cup IS(R_{\text{Right}})$ recursively
 - 5 **return** $\max\{|IS(R_{\text{Median}})|, |IS(R_{\text{Left}}) \cup IS(R_{\text{Right}})|\}$
-

3 Recurrence Relation and analysis

The total running time of the algorithm is bounded by:

$$T(n) \leq 2T(n/2) + O(n)$$

$$T(n) = O(n \log n)$$

Reference. Lecture Notes on Advanced Algorithms and Complexity.